Implementation (Proposed)

1. Tools, Frameworks, and Programming Languages Used

1. Python for backend development and running the trained model.
2. HTML, CSS, JavaScript for web interface or React Native for mobile application.
3. Frameworks: Flask or FastAPI for building API between backend and frontend.
4. Python libraries: Scikit-learn for traditional models (Naive Bayes, SVM), TensorFlow/Keras or PyTorch for deep learning models (LSTM, CNN), NLTK or spaCy for text preprocessing.

2. System Design and Workflow Diagrams

1. Backend server runs the trained model.
2. Frontend (web page or mobile app) sends input messages to the backend via API.
3. Backend processes the data, predicts spam or ham, and returns the result to the frontend.

Proposed Workflow Diagrams

1. System Architecture Diagram: Shows main components (Frontend, Backend, Database, ML model).
2. Data Flow Diagram: Shows data movement from input messages through preprocessing, feature extraction, model prediction, and output.
3. Model Training Workflow Diagram: Shows data collection, preprocessing, training, validation, and evaluation.
4. Prediction Workflow Diagram: Shows how new messages are processed to produce predictions.